

Assignment 1: CSCI433/933 - Machine learning algorithms and applications

Bridging Statistical Learning and Neural Networks for Regression

Instructor: Professor Philip O. Ogunbona
School of Computing and IT
University of Wollongong, Australia

Due date: April 25, 2026

1 Objective

This assignment investigates linear regression through both statistical learning and neural network perspectives, with emphasis on theoretical equivalence, regularisation, and model behaviour.

Students will:

- Establish formal connections between classical and neural approaches
- Conduct rigorous experimental evaluation
- Use Generative AI critically as a reasoning tool
- Think mathematically in the presence of an intelligent but unreliable collaborator (GenAI)

The primary emphasis of this assignment is on conceptual and mathematical understanding, supported by experiments rather than replaced by them.

2 Mathematical Foundations (Mandatory)

Students must derive and present the following:

1. Equivalence between ridge regression and L2-regularised linear neural networks
2. Geometric interpretation of L1 regularisation and sparsity
3. Discussion of dropout as implicit regularisation

2.1 Use of Generative AI in Mathematical Derivations

Generative AI may be used to assist in exploring, drafting, and refining mathematical derivations in this assignment. However, students are reminded that the purpose of this component is not merely to obtain a derivation, but to demonstrate mathematical understanding through verification, reconstruction, and critical evaluation.

If Generative AI is used in any derivation, students must:

- include the relevant prompt–response pair(s) in the Appendix;
- clearly indicate which part of the derivation was assisted by Generative AI;
- verify the correctness of the derivation step by step;
- annotate or explain the key transitions in the derivation in their own words;
- reconstruct the derivation in a coherent mathematical form in the main body of the report;
- identify any omission, ambiguity, unjustified assumption, or error in the AI-generated derivation, where applicable.

Students will be assessed not on whether Generative AI was used, but on the quality of their mathematical validation and interpretation of the derivation. A derivation that is mathematically correct but presented without evidence of verification, annotation, or independent reconstruction will receive limited marks.

Where a Generative AI system produces an incomplete, unclear, or non-rigorous derivation, students are expected to identify this and correct it using appropriate mathematical reasoning and academic references where necessary.

Students should assume that AI-generated derivations may contain subtle errors, omissions, or unjustified steps unless proven otherwise.

3 Dataset

Bike Sharing Dataset (UCI Repository).

Dataset Source:

<https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset>

Students must:

- Perform both random and time-based splits
- Analyse statistical properties and temporal dependencies

3.1 Feature Subset

To ensure consistency and comparability across submissions, students must use the following subset of features:

Feature	Description
temp	Normalized temperature in Celsius
humidity	Normalized humidity
windspeed	Normalized wind speed
season	Encoded as 1–4: 1 (winter), 2 (spring), 3 (summer), 4 (autumn)
holiday	Binary indicator: 1 if holiday, 0 otherwise
workingday	Binary indicator: 1 if working day, 0 otherwise
hour	Hour of day (0–23)

The target variable is:

- `bikes_rented`

Students must use this feature set as the baseline representation. Any additional feature engineering (e.g., polynomial terms or interactions) must be derived explicitly from these features and clearly justified.

4 Experimental Protocol

- Use k-fold cross-validation (k=5)
- Report mean $\pm$ standard deviation
- Perform at least 3 independent runs

Use cross-validation on the training partition for hyperparameter selection. Report final performance on a held-out test partition.

Independent runs are required for models involving stochastic training (e.g., neural networks). For deterministic models, repeated runs are not required.

For the time-based experiment, preserve chronological order and explain the split boundary. Comment on whether the conclusions drawn from random splits remain stable under time-based evaluation, and explain any differences.

5 Tasks

Marks within each component will be awarded **primarily** based on:

- correctness of implementation,
- adherence to experimental protocol,
- depth and clarity of analysis,
- quality of justification and interpretation.

5.1 Mathematical Derivation and Validation (15 Marks)

Students must present and critically evaluate key mathematical derivations related to the models used in this assignment.

This includes:

- Derivation of the equivalence between ridge regression and L2-regularised linear neural networks (6 marks)
- Explanation of sparsity induced by L1 regularisation (Lasso) (4 marks)
- Critical discussion of dropout as a form of implicit regularisation (5 marks)

For each derivation, students must:

- present a clear and mathematically coherent derivation;
- verify and annotate key steps;
- reconstruct the derivation in their own words;

- identify any ambiguity, omission, or error in AI-assisted derivations (if used).

Marks will be awarded based on:

- correctness and completeness;
- depth of understanding;
- quality of validation and critical analysis.

5.2 Statistical Modelling and Analysis (40 Marks)

- Linear Regression (8 Marks)
- Ridge Regression (with tuning) (8 Marks)
- Lasso Regression (sparsity analysis) (8 Marks)
- Elastic Net (6 Marks)

Analysis Requirements: (10 Marks)

- RMSE, R^2 , MAE
- Residual analysis
- Feature importance discussion - for example:
 - coefficient magnitude after standardisation,
 - coefficient shrinkage patterns,
 - features selected or zeroed by lasso,
 - sensitivity across regularisation strengths.

5.3 Neural Modelling and Analysis (40 Marks)

The linear neural network is used to study theoretical equivalence with linear regression, while the hidden-layer model explores nonlinear representational capacity.

- Linear neural network (baseline equivalence) (8 Marks)
- One hidden-layer network (nonlinear extension) (7 Marks)
- Weight decay analysis (7 Marks)
- Investigate whether dropout is an appropriate regularisation mechanism for this problem setting, and justify your conclusion empirically and conceptually. (10 Marks)

Feature Engineering: (8 Marks)

- Polynomial features
- Interaction terms
- Impact on conditioning and generalisation

Students should analyse how feature engineering affects model performance, stability, and generalisation.

5.4 Generative AI Analysis (20 Marks)

Students must use GenAI tools following structured prompting:

Prompt Types Required:

- Exploratory
- Comparative
- Adversarial
- Verification

Critical Evaluation: Classify errors into:

- Factual errors
- Logical inconsistencies
- Hallucinations
- Oversimplifications

Marks for this section will be allocated as follows:

- AI interaction log quality (8 marks)
- Critical evaluation of AI outputs (6 marks)
- Synthesis and integration into the report (6 marks)

The AI interaction log will be assessed based on:

- quality and relevance of prompts;
- evidence of response-driven refinement;
- quality of selected highlights from AI responses;
- clarity of the key insights that motivated follow-up prompts;
- evidence of critical selection, interpretation, and synthesis rather than raw response dumping.

The appendix will be assessed not as a transcript of AI usage, but as evidence of structured reasoning, selective interpretation, and progressive refinement.

6 Submission

- Code (Python)
- Report (6–8 pages)
- Appendix with prompt-response logs

Core derivations and analysis must appear in the main report. The appendix is for supporting material (e.g., prompt–response logs).

6.1 Use of Academic References

Students must support key technical claims, derivations, and evaluations with appropriate academic references.

The following minimum requirements apply:

- At least four (4) academic references must be included.
- At least two (2) references must be used to validate or critique Generative AI outputs.
- At least one (1) reference must support a mathematical derivation or theoretical claim.

References may include textbooks, peer-reviewed journal articles, or reputable conference papers. Web sources may be used only for dataset provenance or software documentation, not as substitutes for academic references supporting technical claims.

Citations should be used selectively to support important claims, not inserted generically. Submissions that include references without clear linkage to the discussion will receive limited credit.

6.2 Appendix: Generative AI Interaction Logs

Students must include a record of their Generative AI interactions in the Appendix. The purpose of this appendix is to demonstrate structured reasoning and iterative refinement, not to reproduce raw AI outputs in full.

The following requirements apply:

- All prompts must be included verbatim.
- AI responses should **not** be included in full. Instead, students must provide:
 - a concise summary or selected excerpts of the response (highlights), and
 - the key insights, issues, or limitations identified in the response.
- Students must explicitly state how each response informed the subsequent prompt or refinement.

The appendix should clearly demonstrate a process of:

- critical evaluation of AI outputs,
- identification of gaps or weaknesses,
- and progressive refinement of understanding.

Submissions that include large volumes of unprocessed AI-generated text without selection, interpretation, and synthesis will receive limited credit.

7 Note on Academic Integrity

Submissions dominated by unverified AI-generated content will receive low marks.

The following are examples of what is not acceptable:

- submitting AI-generated derivations without verification,
- omitting used prompts,
- citing references not actually consulted,
- presenting AI-generated prose as original critical analysis.

Appendix A: Example Generative AI Prompts

This appendix provides example prompts illustrating the expected quality and diversity of Generative AI usage in this assignment. Students are expected to refine and extend such prompts iteratively.

A.1 Exploratory Prompts

These prompts aim to build initial conceptual understanding.

- Explain the relationship between ridge regression and L2 regularisation in neural networks.
- What is the intuition behind L1 regularisation leading to sparse solutions?
- How does overfitting manifest in linear regression models?

A.2 Comparative Prompts

These prompts are used to analyse differences between methods.

- Compare ridge regression and lasso in terms of bias–variance trade-off and feature selection.
- What are the similarities and differences between linear regression and a linear neural network trained with gradient descent?
- Compare weight decay and dropout as regularisation techniques.

A.3 Adversarial Prompts

These prompts challenge the AI system and test robustness.

- A claim is made that dropout always improves regression performance. Is this true? Provide counterexamples.
- Is it correct to say that ridge regression always outperforms lasso? Critically evaluate this claim.
- Can a linear neural network represent nonlinear relationships? Explain any limitations.

A.4 Verification Prompts

These prompts aim to validate derivations and technical claims.

- Derive the solution for ridge regression and show how it differs from ordinary least squares.
- Show that L2 regularisation in a neural network corresponds to adding a penalty term to the loss function.
- Verify whether the following statement is correct: “Dropout approximates model averaging.” Provide justification.

High-quality submissions will demonstrate that successive prompts are informed by critical evaluation of earlier responses and lead to deeper or more precise understanding.

A.5 Iterative Dialogue Example (Response-Driven Refinement)

Effective use of Generative AI involves iterative interaction, where each response is critically evaluated and used to formulate a more precise or deeper follow-up prompt.

Initial Prompt:

Explain ridge regression.

AI Response (excerpt):

Ridge regression is a technique that adds an L2 penalty to the least squares objective to prevent overfitting.

Student Reflection:

- The response is conceptually correct but lacks mathematical detail.
- No derivation or explicit formulation is provided.

Follow-up Prompt:

Provide the full mathematical formulation of ridge regression and derive the closed-form solution.

AI Response (excerpt):

The ridge regression solution is $(X^T X + \lambda I)^{-1} X^T y$.

Student Reflection:

- The expression is presented as a result, not derived.
- Intermediate steps are missing.
- It is unclear under what assumptions this solution holds.

Refined Prompt:

Derive ridge regression step by step starting from the regularised least squares objective. Clearly state assumptions and justify each step.

AI Response (excerpt):

Starting from the objective $\|y - Xw\|^2 + \lambda\|w\|^2$, we take derivatives with respect to w and set to zero...

Student Reflection:

- The derivation is now clearer, but assumptions about invertibility are not discussed.

Final Prompt:

Extend the derivation to the case where $X^T X$ is not invertible. Discuss how ridge regression ensures a unique solution.

Key Insight:

- Each prompt is informed by limitations in the previous response.
- The interaction demonstrates progressive refinement of understanding.
- This process mirrors mathematical reasoning and research inquiry.

Appendix B: How to Approach

This assignment is not only about implementing models. It is designed to assess how you:

- reason mathematically,
- design and interpret experiments,
- and use Generative AI critically.

B.1 Start with Understanding, Not Code

Before implementing anything:

- Understand the dataset (distributions, trends, correlations).
- Reflect on what linear regression assumes.
- Think about how regularisation changes model behaviour.

Your implementation should follow from understanding, not the other way around.

B.2 Treat Mathematical Derivation as Validation

You may use Generative AI to assist with derivations, but:

- Do not accept derivations at face value.
- Check each step carefully.
- Rewrite the derivation in your own words.

Ask yourself:

“Do I understand why each step is valid?”

B.3 Design Experiments Carefully

Follow a disciplined workflow:

- Use cross-validation for tuning.
- Evaluate on a separate test set.
- Compare random vs time-based splits.

Do not just report results. Interpret them:

- Why does one model perform better?
- How does regularisation affect behaviour?
- Are results stable across splits?

B.4 Use Generative AI as a Thinking Partner

Do not use AI to generate final answers. Use it to:

- explore ideas,
- challenge assumptions,
- refine your understanding.

A good workflow is:

1. Ask an initial question.
2. Critically evaluate the response.
3. Identify gaps or weaknesses.
4. Ask a more precise follow-up question.

Each prompt should be motivated by the previous response.

B.5 Be Critical of AI Outputs

Assume AI responses may contain:

- missing steps,
- incorrect assumptions,
- vague or incomplete explanations.

Your task is to:

- identify these issues,
- correct them,
- validate using academic references.

B.6 Use References Strategically

References are not for decoration. Use them to:

- confirm derivations,
- support claims,
- resolve uncertainty in AI outputs.

Ask yourself:

“What evidence supports this claim?”

Focus on Insight, Not Volume

High-quality submissions:

- explain *why*, not just *what*,
- connect theory and experiments,
- demonstrate clear reasoning.

Low-quality submissions:

- list results without interpretation,
- copy AI outputs without critique,
- include code without explanation.

B.7 Final Advice

Think of this assignment as:

an exercise in reasoning with an intelligent but imperfect assistant.

Your goal is not to show that you can use tools, but that you can:

- think clearly,
- question results,
- and justify your conclusions.

Appendix C: Starter Code (Optional)

A minimal starter code snippet is provided below to assist with basic data loading, preprocessing, and library setup. Its purpose is to reduce implementation overhead, particularly for students less familiar with the programming environment.

Students are not required to use the starter code.

If used, students must:

- understand and be able to explain all components of the code;
- modify and extend it as necessary to support their experimental design;
- ensure that their final implementation reflects the requirements of the assignment, rather than the structure of the starter code.

Use of the starter code without modification, analysis, or understanding will receive limited credit.

The starter code does not constitute a complete solution and does not define the experimental protocol, modelling approach, or evaluation strategy.

```

1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.preprocessing import StandardScaler
4
5  # -----
6  # Minimal starter code for Assignment 1
7  # -----
8  # Purpose:
9  #   - Load the dataset
10 #   - Select the required baseline features
11 #   - Create training and test partitions
12 #   - Standardise the input features
13 #
14 # This code is intentionally minimal.
15 # It does NOT implement:
16 #   - cross-validation
17 #   - hyperparameter tuning
18 #   - regression models
19 #   - neural networks
20 #   - evaluation metrics
21 #   - time-based splitting
22 # Students must design and justify those components themselves.
23 # -----
24
25 # Load the dataset
26 df = pd.read_csv("bike_rental_data.csv")
27
28 # Required baseline feature set
29 feature_columns = [
30     "temp",
31     "humidity",
32     "windspeed",
33     "season",
34     "holiday",
35     "workingday",
36     "hour"
37 ]
38
39 target_column = "bikes_rented"
40
41 # Separate inputs and target
42 X = df[feature_columns].copy()
43 y = df[target_column].copy()
44
45 # Random train-test split (baseline only)
46 # Students must also implement a time-based split separately
47 X_train, X_test, y_train, y_test = train_test_split(
48     X,
49     y,
50     test_size=0.2,
51     random_state=42
52 )
53

```

```
54 # Standardise input features using statistics from the training set only
55 scaler = StandardScaler()
56 X_train_scaled = scaler.fit_transform(X_train)
57 X_test_scaled = scaler.transform(X_test)
58
59 # Simple sanity checks
60 print("Training samples:", X_train.shape[0])
61 print("Test samples:", X_test.shape[0])
62 print("Number of features:", X_train.shape[1])
63 print("Feature columns:", feature_columns)
```
